DESIGN AND ANALYSIS OF ALGORITHMS

Assignment 1 (CO1)

Design, Analysis and Validation of an Efficient Search Strategy for a Large-Scale Student Record System

Submitted by	N D YOGESHWAR
Registration number	192465067
Dataset seed	5067 (last four digits of the registration number)
Third algorithm	Binary Search Tree (group D)
Programming language	Python 3.11
GitHub repository	https://github.com/yogeshwardev/DAA-A1-192465067-N-D-YOGESHWAR

Abstract

This report studies the problem of finding a student record from its registration number when the database grows from one thousand to one million records. Three different strategies were designed: linear search, recursive binary search on a sorted array, and search in a binary search tree. For each of them the basic operation was identified, the running time was derived mathematically for the best, average and worst cases, and the recurrence of the recursive algorithm was formed and solved. The algorithms were then implemented, run on datasets of 1,000, 10,000, 100,000 and 1,000,000 records and the measured comparison counts and times were compared with the theory. Finally the design was re-examined for a workload of 50,000 searches per hour on one million records with one batch of new records per day.

The measured comparison counts followed the derived formulae closely. Binary search on a sorted array was found to be the most suitable design, and for the changed workload of Task 12 it is kept with a daily merge of new records in place of a full sort.

Contents

Task 1. Design of the algorithms

Task 2. Basic operation

Task 3. Time-complexity derivation

Task 4. Best, average and worst case

Task 5. Asymptotic analysis

Task 6. Formulation of the recurrence

Task 7. Solution of the recurrence

Task 8. Implementation and testing

Task 9. Experimental performance

Task 10. Theory compared with experiment

Task 11. Final design recommendation

Task 12. Day-4 design challenge

Conclusion and references

Task 1. Design of the Algorithms

The registration numbers are unique integers that arrive in no particular order. Three strategies were chosen so that each of them treats the data in a different way: examining every record, reducing the problem by half after sorting, and restructuring the records into a tree. Linear and binary search are compulsory, and the binary search tree was selected as the third algorithm for roll-number group D.

1.1 Algorithm 1: linear search (brute force, iterative)

The records are scanned from the first position, and each registration number is compared with the key until a match is found or the list ends. No preprocessing is needed, because the records can stay in the order of arrival. This method was selected because it is the simplest possible solution and gives the baseline against which the other two are judged.

```
LinearSearch(data[0..n-1], key)
    for i = 0 to n-1
        if data[i] == key
            return i
    return -1
```

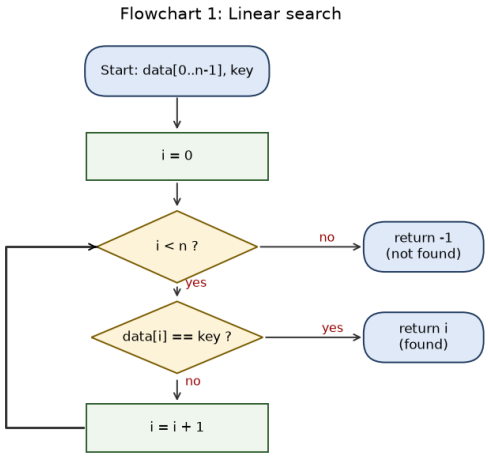


Figure 1.1: Flowchart of linear search

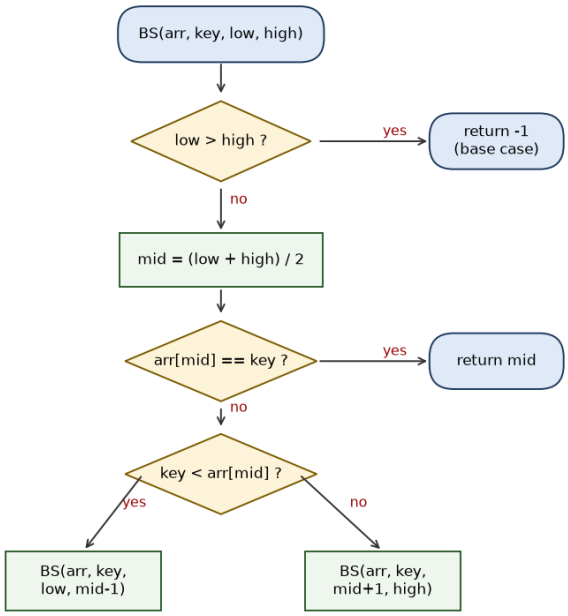
1.2 Algorithm 2: binary search (decrease and conquer, recursive)

When the array is sorted, the key is compared with the middle element. If they are equal the search ends; otherwise the key can lie only in one half, so the other half is discarded and the same step is applied to the remaining half. The records must be sorted once before the first search, and this preprocessing has a cost that must be counted. The method was selected to find out whether this one-time cost is worth paying, and it is the recursive approach required by the assignment.

```
Sort(data)                              // preprocessing
BinarySearch(arr, key, low, high)
    if low > high
        return -1                       // base case
```

```
mid = (low + high) / 2
if arr[mid] == key
    return mid
if key < arr[mid]
    return BinarySearch(arr, key, low, mid - 1)
return BinarySearch(arr, key, mid + 1, high)
```

Flowchart 2: Recursive binary search (sorted array)



N D YOGESHWAR | 192465067

Figure 1.2: Flowchart of recursive binary search

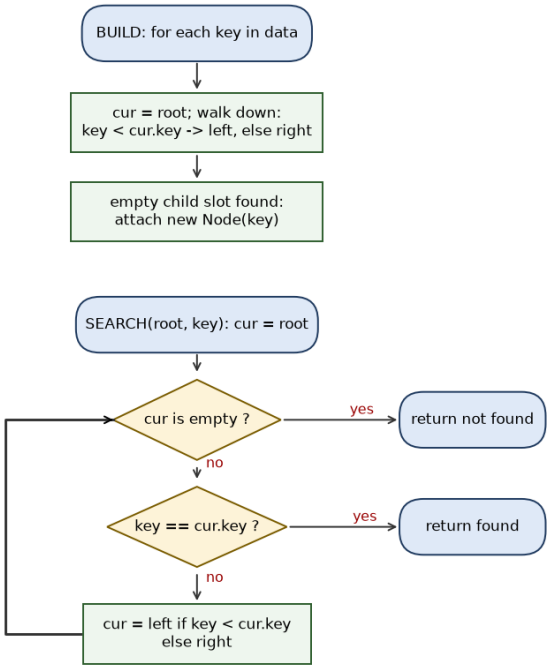
1.3 Algorithm 3: binary search tree (transform and conquer, iterative)

In this approach the representation of the data is changed. Every record becomes a node of a tree; keys smaller than the key of a node are placed in its left subtree and larger keys in its right subtree. A search begins at the root and moves to the left or the right child at every node, so only one path from the root is followed. The preprocessing consists of inserting all records into the tree in the order of arrival. This strategy differs from binary search, because the structure is grown from the data and not obtained by sorting, and it also allows a single record to be added without sorting again. Both the build and the search were written with loops, so that a very deep tree cannot exceed the recursion limit of Python.

```
Build(data)                              // preprocessing
    root = empty
    for each key in data
        cur = root
        while true
            if key < cur.key
                if cur.left is empty: cur.left = Node(key); break
                cur = cur.left
            else
                if cur.right is empty: cur.right = Node(key); break
                cur = cur.right
```

```
BSTSearch(root, key)
    cur = root
    while cur is not empty
        if key == cur.key
            return found
        if key < cur.key: cur = cur.left
        else:             cur = cur.right
    return not found
```

Flowchart 3: BST build (preprocessing) and search



N D YOGESHWAR | 192465067

Figure 1.3: Flowchart of BST construction and search

	Linear search	**Binary search**	**BST search**
Design strategy	Brute force	Decrease and conquer	Transform and conquer
Implementation	Iterative	Recursive	Iterative
Preprocessing	None	Sorting the records	Inserting all records into a tree
Extra memory	None	A sorted copy	One node (two pointers) per record

Table 1.1: Summary of the three approaches

Task 2. Basic Operation

In all three algorithms the time taken is governed by the number of comparisons between a stored registration number and the key. The other statements, such as changing an index or finding the middle position, are executed at most once for each comparison, so they change only the constant factor and not the way the running time grows with n. For this reason the comparison was taken as

the basic operation.

In linear search the basic operation is the test `data[i] == key` in the body of the loop. It is executed once for every position that is visited, so its count equals the position of the key or n when the key is absent. If n becomes ten times larger, a search that has to travel through the list takes ten times as many comparisons.

algo1_linear_search.py (basic operation highlighted)

```
 1  def linear_search(data, key):
 2      comparisons = 0
 3      for i in range(len(data)):
 4          comparisons += 1
 5          if data[i] == key:
 6              return i, comparisons
 7      return -1, comparisons
```

N D YOGESHWAR | Reg No 192465067

Figure 2.1: Basic operation of linear search (highlighted line)

In recursive binary search the basic operation is the comparison of the key with `arr[mid]`, which is made once in every recursive call. Each call removes half of the remaining records, so the number of calls is the number of times n can be halved, a quantity that grows very slowly. The counter in the program is increased at exactly the place where this comparison is made.

algo2_binary_search.py (basic operation highlighted)

```
 1  def preprocess(data):
 2      return sorted(data)
 3
 4
 5  def binary_search(arr, key, low=0, high=None, _c=None):
 6      if high is None:
 7          high = len(arr) - 1
 8      if _c is None:
 9          _c = [0]
10      if low > high:
11          return -1, _c[0]
12      mid = (low + high) // 2
13      _c[0] += 1
14      if arr[mid] == key:
15          return mid, _c[0]
16      if key < arr[mid]:
17          return binary_search(arr, key, low, mid - 1, _c)
18      return binary_search(arr, key, mid + 1, high, _c)
19
20
21  def binary_search_iterative(arr, key):
22      low, high, comparisons = 0, len(arr) - 1, 0
23      while low <= high:
24          mid = (low + high) // 2
25          comparisons += 1
26          if arr[mid] == key:
27              return mid, comparisons
28          if key < arr[mid]:
29              high = mid - 1
30          else:
31              low = mid + 1
32      return -1, comparisons
```

N D YOGESHWAR | Reg No 192465067

Figure 2.2: Basic operation of binary search (highlighted lines)

In the tree search the basic operation is the comparison of the key with `cur.key` at every node that is visited. The number of comparisons equals the depth of the node where the search stops, so it depends on the height of the tree, and that in turn depends on the order in which the records were

inserted. During construction the comparison `key < cur.key` plays the same role.

algo3_bst_search.py (basic operation highlighted)

```
 1  class Node:
 2      __slots__ = ("key", "left", "right")
 3
 4      def __init__(self, key):
 5          self.key = key
 6          self.left = None
 7          self.right = None
 8
 9
10  def build_bst(data):
11      root = None
12      for key in data:
13          if root is None:
14              root = Node(key)
15              continue
16          cur = root
17          while True:
18              if key < cur.key:
19                  if cur.left is None:
20                      cur.left = Node(key)
21                      break
22                  cur = cur.left
23              else:
24                  if cur.right is None:
25                      cur.right = Node(key)
26                      break
27                  cur = cur.right
28      return root
29
30
31  def bst_search(root, key):
32      comparisons = 0
33      cur = root
34      while cur is not None:
35          comparisons += 1
36          if key == cur.key:
37              return True, comparisons
38          cur = cur.left if key < cur.key else cur.right
39      return False, comparisons
40
41
42  def height(root):
43      if root is None:
44          return 0
45      level, h = [root], 0
46      while level:
47          h += 1
48          level = [c for n in level for c in (n.left, n.right) if c is not None]
49      return h
```

N D YOGESHWAR | Reg No 192465067

Figure 2.3: Basic operation of BST search (highlighted lines)

Task 3. Derivation of the Time-Complexity Function

The input size n is the number of student records. C(n) denotes the number of key comparisons made by one search. The cost of preprocessing is added separately, because the assignment requires it to be part of the analysis.

3.1 Linear search

If the key is at position i, the loop makes exactly i comparisons, and if the key is absent it makes n. Assuming that the key is equally likely to be at any of the n positions, $C_{avg}(n) = (1 + 2 + ... + n)/n = [n(n + 1)/2]/n = (n + 1)/2$. Therefore $C_{best} = 1$, $C_{avg} = (n + 1)/2$ and $C_{worst} = n$. There is no preprocessing, so the total cost of q searches is between $q \cdot n/2$ and $q \cdot n$, and the cost of one search is $\Theta(n)$.

3.2 Binary search

After one comparison at most n/2 records remain, after two at most n/4, and after k comparisons at most $n/2^k$. The search has to stop when the range becomes empty, that is when $n/2^k < 1$, which gives $k > \log_2 n$. Hence $C_{worst}(n) = \text{floor}(\log_2 n) + 1$. For n = 1,000,000 this equals 19 + 1 = 20, and the largest value recorded by the program in Task 9 is also 20.

For the average successful search the comparison tree can be used. One element is found with 1 comparison, two elements with 2, four with 3, and in general 2^{j-1} elements with j comparisons. For $n = 2^m - 1$ the total is $S = \Sigma\, j \cdot 2^{j-1}$ (j = 1 to m) $= (m - 1)2^m + 1$, so $C_{avg} = S/n \approx m - 1 \approx \log_2 n - 1$.

The preprocessing is the sorting of n records, which costs $\Theta(n \log n)$ for a comparison sort. The overall cost of q searches is therefore $n \log_2 n + q \log_2 n = (n + q) \log_2 n$. Linear search needs about $q \cdot n/2$ comparisons for the same searches, so binary search becomes cheaper after only a few searches when n is large; Task 11 gives the number obtained from the measurements.

3.3 Binary search tree

A search that ends at a node of depth d makes d comparisons, with the root at depth 1, so the cost depends on the shape of the tree. When the keys are inserted in random order, the expected depth of a node is $D(n) = 2(1 + 1/n)H_n - 4$, where H_n is the n-th harmonic number and $H_n \approx \ln n + 0.577$. This gives $D(n) \approx 2 \ln n - 2.85 = 1.39 \log_2 n - 2.85$, about 39 percent more than the best possible value. For n = 1,000,000 the formula gives $1.39 \times 19.93 - 2.85 \approx 24.9$, and the average measured for stored keys was 25.0. When the keys arrive in sorted order the tree degenerates into a chain and $C_{worst} = n$.

The insertion of the i-th key costs about as many comparisons as the depth of its position, so building the tree costs $\Sigma\, 2 \ln i \approx 2n \ln n = \Theta(n \log n)$ on the average, and $\Sigma\, i = n(n + 1)/2 = \Theta(n^2)$ for sorted input. The overall cost of q searches on random input is about $2n \ln n + q \cdot 1.39 \log_2 n$.

	Preprocessing	**Search, average**	**Search, worst**	**n records and q searches**
Linear	0	(n + 1)/2	n	q·n/2 on average
Binary	n log n	log n - 1	floor(log n) + 1	n log n + q log n
BST	2n ln n on average, $n^2/2$ worst	1.39 log n - 2.85	n	2n ln n + q·1.39 log n

Table 3.1: Derived cost functions

Task 4. Best, Average and Worst Case

The cases arise from the position of the key in the data and, for the tree, from the order in which the records were inserted. Each case was also measured with the comparison counter on 100,000 records (Figure 4.1).

For linear search the best case occurs when the key is the first record, and a single comparison is enough. The worst case occurs when the key is the last record or is not present at all, and then all n records are examined, which is 100,000 comparisons for this dataset. If every position is equally likely, the search stops on average near the middle, (n + 1)/2, and a random sample of 2,000 stored keys gave about 50,576 against the theoretical 50,000. A search for a registration number that does not exist is always the worst case, which is important because such searches are common in a real system.

For binary search the best case is a key lying exactly at the first midpoint, one comparison. The worst case is a missing key or a key on the last level of the comparison tree, floor($\log_2$ n) + 1 = 17 comparisons for n = 100,000. Half of all the records are on the last level of that tree, and this is why the average successful search, about $\log_2$ n - 1 ≈ 15.7 comparisons, is only one or two comparisons below the worst case.

For the tree the best case is a key at the root. In the average case the tree is built from records in random order, so it is reasonably balanced and a typical node is about 1.39 $\log_2$ n levels deep (about 21.5 was measured). The deepest node of this tree, which is the worst case for it, has depth 44. The real worst case occurs when the records arrive in sorted order, because then every new key goes to the right of the previous one and the tree becomes a linked list. This was demonstrated with 2,000 sorted records: the height was 2,000 and the last key needed 2,000 comparisons. A tree built in this way is therefore fast only when the insertion order is random.

```
C:\DAA-A1\src>  python benchmark.py --cases
======================================================================
Reg No: 192465067 | Name: N D YOGESHWAR
Date/Time: 05-10-2026 23:23:18
TASK 4 - BEST / AVERAGE / WORST CASE COUNTERS  (n = 100,000)
======================================================================

Linear search (basic op = comparison data[i] == key)
  best    : key is first element             ->         1 comparison
  average : successful, random key (mean of 2000) ->  50,576.2  (theory (n+1)/2 = 50,000.5)
  worst   : key is last element              ->   100,000 comparisons
  worst   : key not present                  ->   100,000 comparisons

Binary search (basic op = comparison with arr[mid])
  best    : key sits at the first midpoint   ->         1 comparison
  average : successful, random key           ->     15.70  (theory ~ log2 n - 1 = 15)
  worst   : deepest hit / key not present    ->        17 comparisons  (theory floor(log2 n)+1 = 17)

BST search on randomly ordered input (basic op = comparison with node.key)
  best    : key is the root                  ->         1 comparison
  average : successful, random key           ->     21.54  (theory ~ 1.39 log2 n = 22.2)
  worst   : deepest node (tree height)       ->        44 comparisons

BST worst-case demonstration: 2,000 records inserted in SORTED order
  height = 2,000; searching the last key costs 2,000 comparisons (= n)

                                                  N D YOGESHWAR | Reg No 192465067
```

Figure 4.1: Counter output for the best, average and worst cases (n = 100,000)

Task 5. Asymptotic Analysis

Algorithm	Best case	Average case	Worst case	Remark
Linear search	Ω(1)	Θ(n)	Θ(n)	O(n) overall
Binary search	Ω(1)	Θ(log n)	Θ(log n)	after an O(n log n) sort
BST search	Ω(1)	Θ(log n)	Θ(n)	Θ(log n) only for random order

Table 5.1: Asymptotic behaviour

The lower bound Ω(1) is the same for all three, since the key may always be at the first place that is examined. The worst case is the one the system must be designed for. Preprocessing is Θ(n log n)

for binary search (sorting) and for the tree built from random input, and it is absent for linear search.

n	$\log_2 n$	$n \log_2 n$	Linear: n	Binary: $\log_2 n$	BST: $1.39 \log_2 n$
1,000	9.97	9,966	1,000	10.0	13.9
10,000	13.29	132,877	10,000	13.3	18.5
100,000	16.61	1,660,964	100,000	16.6	23.1
1,000,000	19.93	19,931,569	1,000,000	19.9	27.7

Table 5.2: Growth of the functions from 10^3 to 10^6 records

When the database grows from 1,000 to 1,000,000 records, which is a thousand times more data, the work of linear search also grows a thousand times, so a search that is unnoticeable at one thousand records becomes a delay of a few hundred milliseconds in this program at one million. For binary search $\log_2 n$ rises only from about 10 to about 20, so a thousand times more data costs just twice as many comparisons, and the tree behaves in the same way with a constant factor of about 1.39. A growth like n cannot be accepted for a large database, whereas a growth like log n can, and the price paid for it is the one-time preprocessing of order n log n.

Task 6. Formulation of the Recurrence Relation

The recurrence is formed for the recursive algorithm, which is the recursive binary search.

The size of the original problem is n = high - low + 1, the number of records still in the range. After the key has been compared with `arr[mid]` the function continues either on the left part (low to mid - 1) or on the right part (mid + 1 to high), and each of these contains at most floor(n/2) records. Hence the subproblem has size n/2. Only one of the two recursive calls in the code is executed for a given call, so the number of recursive calls is one. The work done outside the call consists of computing mid and making one comparison, which is a constant amount, taken as 1 basic operation. The recursion stops when low > high; the range is then empty and no comparison is made, so T(0) = 0. A range of a single record makes one comparison and then calls itself on two empty ranges, so T(1) = 1.

Combining these facts, the worst case is described by

```
T(n) = T(n/2) + 1      for n > 1
T(1) = 1
```

In the program the term '+ 1' is the statement `c[0] += 1`, and the term T(n/2) is the single `return binary_search(...)` on one half of the range.

Task 7. Solution of the Recurrence Relation

The iteration (backward substitution) method is applicable because the recurrence refers to only one smaller instance, T(n/2), which can be replaced repeatedly. Let $n = 2^k$.

```
T(n) = T(n/2) + 1
     = [T(n/4) + 1] + 1          = T(n/4) + 2
     = [T(n/8) + 1] + 2          = T(n/8) + 3
     ...
     = T(n / 2^i) + i
```

```
The recursion ends when n / 2^i = 1, i.e. i = log2 n, so
T(n) = T(1) + log2 n = 1 + log2 n
```

The result can be checked with the Master Theorem. The recurrence has the form $T(n) = aT(n/b) + f(n)$ with $a = 1$, $b = 2$ and $f(n) = 1$. Since $n^{\log_b a} = n^0 = 1$ and $f(n) = \Theta(1)$, the second case of the theorem applies and $T(n) = \Theta(\log n)$.

The result can also be proved by substitution. Suppose $T(n) \leq c \log_2 n + 1$ for $n \geq 2$. Then $T(n) = T(n/2) + 1 \leq c \log_2(n/2) + 1 + 1 = c \log_2 n - c + 2$, which is at most $c \log_2 n + 1$ when $c \geq 1$. The base case $T(1) = 1$ satisfies the assumption.

Thus $T(n) = 1 + \log_2 n = \Theta(\log n)$, in agreement with the count $\text{floor}(\log_2 n) + 1$ of Task 3, which is the exact value when n is not a power of two. The result shows that every doubling of the database adds only one more comparison: one million records need about 20 comparisons and two million need about 21. The depth of the recursion is the same number, so there is no danger of exhausting the stack.

Task 8. Implementation and Testing

The three algorithms were implemented in Python in the files `algo1_linear_search.py`, `algo2_binary_search.py` and `algo3_bst_search.py`. The dataset generator uses the random seed 5067, which are the last four digits of the registration number 192465067, and draws unique nine-digit numbers between 192,000,000 and 194,999,999. Every dataset contains 192465067 and never contains 192465068, and these two numbers are included in the test queries. All algorithms receive the same data and the same list of 202 queries, which consists of 101 stored keys and 101 absent keys.

The test cases are the first, last, middle, smallest and largest keys, three kinds of absent key, a list with a single element and an empty list. For 1,000, 10,000 and 100,000 records every query is also checked against a Python set to confirm that the three algorithms return the correct answer. For 1,000,000 records the benchmark itself checks that the three algorithms agree on every query.

```
C:\DAA-A1\src>  python benchmark.py --test
======================================================================
Reg No: 192465067 | Name: N D YOGESHWAR
Date/Time: 05-10-2026 23:23:14
TASK 8 - CORRECTNESS TESTS
======================================================================
test case              key    linear  binary     BST  expected  result
first element           42      True    True    True  True      PASS
last element            61      True    True    True  True      PASS
middle value            19      True    True    True  True      PASS
smallest value           3      True    True    True  True      PASS
largest value           88      True    True    True  True      PASS
absent (low)             1     False   False   False  False     PASS
absent (high)           99     False   False   False  False     PASS
absent (between)        20     False   False   False  False     PASS
single element           5      True    True    True  True      PASS
single element           6     False   False   False  False     PASS
empty list               5     False   False   False  False     PASS
empty list               6     False   False   False  False     PASS
----------------------------------------------------------------------
n=  1,000: 202 queries, mismatches=0; own reg 192465067 found by (lin,bin,bst)=(True, True, True); reg+1 192465068 found=(False, False, False)  -> PASS
n= 10,000: 202 queries, mismatches=0; own reg 192465067 found by (lin,bin,bst)=(True, True, True); reg+1 192465068 found=(False, False, False)  -> PASS
n=100,000: 202 queries, mismatches=0; own reg 192465067 found by (lin,bin,bst)=(True, True, True); reg+1 192465068 found=(False, False, False)  -> PASS
----------------------------------------------------------------------
ALL TESTS PASSED

                                                    N D YOGESHWAR | Reg No 192465067
```

Figure 8.1: Output of the correctness tests

The program was run with the commands `python benchmark.py --test` and `python benchmark.py --size n` for the four sizes. The complete instructions are in the README file of the repository.

Task 9. Experimental Performance

For every dataset the program recorded the time for preprocessing, the total time for the 202 searches, the average time for one search, and the average and maximum number of comparisons. The time was measured with `time.perf_counter()`. The comparison counter is inside the search functions, so the times include the small cost of counting, which is the same for all algorithms.

n	Algorithm	Preprocessing (s)	202 searches (s)	Average time per search (µs)	Average comparisons	Maximum comparisons
1,000	Linear search	0.0000	0.0204	100.91	738.7	1,000
1,000	Binary search (recursive)	0.0002	0.0013	6.37	9.4	10
1,000	BST search	0.0022	0.0005	2.35	12.5	21
10,000	Linear search	0.0000	0.2810	1391.17	7545.2	10,000
10,000	Binary search (recursive)	0.0039	0.0019	9.17	12.9	14
10,000	BST search	0.0299	0.0007	3.69	17.5	29
100,000	Linear search	0.0000	4.3595	21581.91	76036.8	100,000
100,000	Binary search (recursive)	0.0858	0.0042	20.97	16.1	17
100,000	BST search	0.6114	0.0017	8.24	21.7	38
1,000,000	Linear search	0.0000	61.0085	302022.45	737490.1	1,000,000
1,000,000	Binary search (recursive)	0.7930	0.0030	14.73	19.4	20
1,000,000	BST search	8.0645	0.0016	7.75	25.6	39

Table 9.1: Measured results (taken from results.csv)

```
C:\DAA-A1\src>  python benchmark.py --size 1000
```

```
======================================================================
Reg No: 192465067 | Name: N D YOGESHWAR
Date/Time: 05-10-2026 23:23:40
TASK 9 - BENCHMARK  n = 1,000 records
======================================================================
dataset generated (seed 5067): 1,000 unique records in 0.002s
queries: 202 (100 found + 100 not found + own reg 192465067 + 192465068)

all three algorithms agree on every query: True

algorithm                  preprocess(s) search tot(s) avg us/query  avg comps max comps
Linear search                     0.0000        0.0204       100.91      738.7      1000
Binary search (recursive)         0.0002        0.0013         6.37        9.4        10
BST search                        0.0022        0.0005         2.35       12.5        21

BST height = 21  (log2 n = 9+)

results saved to results/results.csv
```

```
N D YOGESHWAR | Reg No 192465067
```

Figure 9.1: Program output for n = 1,000 records

```
C:\DAA-A1\src>  python benchmark.py --size 10000
```

```
======================================================================
Reg No: 192465067 | Name: N D YOGESHWAR
Date/Time: 05-10-2026 23:23:41
TASK 9 - BENCHMARK  n = 10,000 records
======================================================================
dataset generated (seed 5067): 10,000 unique records in 0.019s
queries: 202 (100 found + 100 not found + own reg 192465067 + 192465068)

all three algorithms agree on every query: True

algorithm                  preprocess(s) search tot(s) avg us/query  avg comps max comps
Linear search                     0.0000        0.2810      1391.17     7545.2     10000
Binary search (recursive)         0.0039        0.0019         9.17       12.9        14
BST search                        0.0299        0.0007         3.69       17.5        29

BST height = 32  (log2 n = 13+)

results saved to results/results.csv
```

```
N D YOGESHWAR | Reg No 192465067
```

Figure 9.2: Program output for n = 10,000 records

```
C:\DAA-A1\src>  python benchmark.py --size 100000

======================================================================
Reg No: 192465067 | Name: N D YOGESHWAR
Date/Time: 05-10-2026 23:23:42
TASK 9 - BENCHMARK  n = 100,000 records
======================================================================
dataset generated (seed 5067): 100,000 unique records in 0.233s
queries: 202 (100 found + 100 not found + own reg 192465067 + 192465068)

all three algorithms agree on every query: True

algorithm                   preprocess(s) search tot(s) avg us/query  avg comps max comps
Linear search                      0.0000        4.3595     21581.91    76036.8    100000
Binary search (recursive)          0.0858        0.0042        20.97       16.1        17
BST search                         0.6114        0.0017         8.24       21.7        38

BST height = 44  (log2 n = 16+)

results saved to results/results.csv

                                                    N D YOGESHWAR | Reg No 192465067
```

Figure 9.3: Program output for n = 100,000 records

```
C:\DAA-A1\src>  python benchmark.py --size 1000000

======================================================================
Reg No: 192465067 | Name: N D YOGESHWAR
Date/Time: 05-10-2026 23:23:47
TASK 9 - BENCHMARK  n = 1,000,000 records
======================================================================
dataset generated (seed 5067): 1,000,000 unique records in 2.991s
queries: 202 (100 found + 100 not found + own reg 192465067 + 192465068)

all three algorithms agree on every query: True

algorithm                   preprocess(s) search tot(s) avg us/query  avg comps max comps
Linear search                      0.0000       61.0085    302022.45   737490.1   1000000
Binary search (recursive)          0.7930        0.0030        14.73       19.4        20
BST search                         8.0645        0.0016         7.75       25.6        39

BST height = 52  (log2 n = 19+)

results saved to results/results.csv

                                                    N D YOGESHWAR | Reg No 192465067
```

Figure 9.4: Program output for n = 1,000,000 records

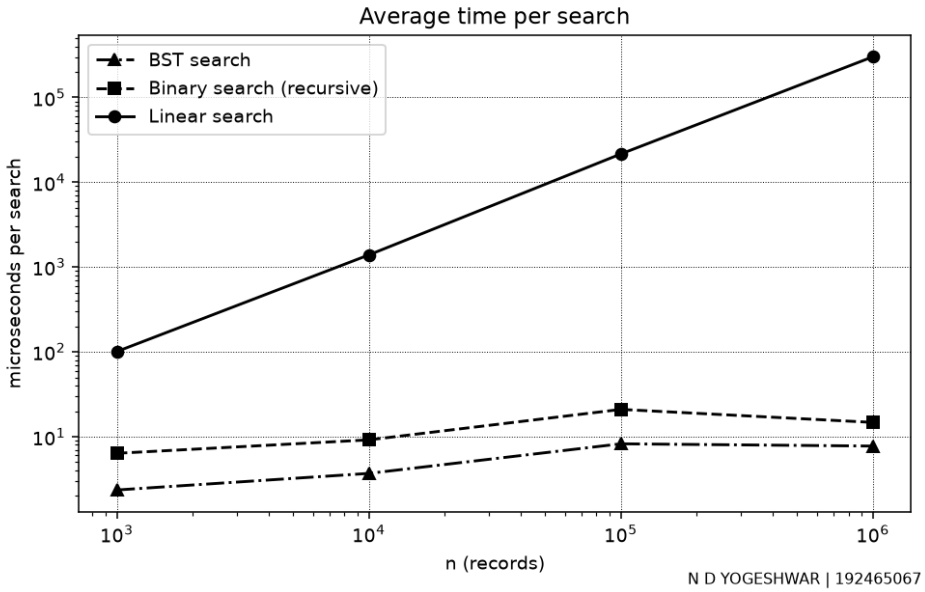


Figure 9.5: Average time per search (logarithmic axes)

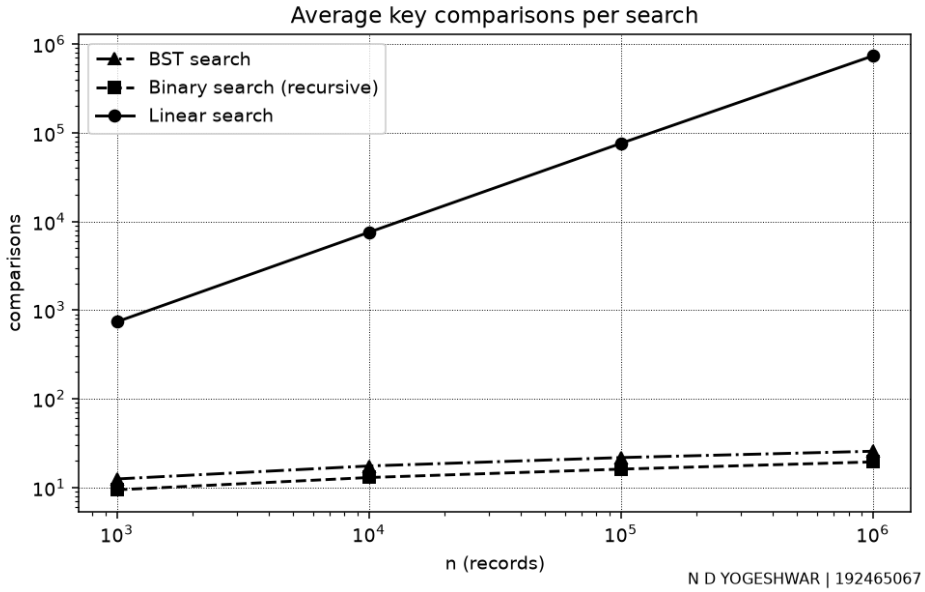


Figure 9.6: Average comparisons per search (logarithmic axes)

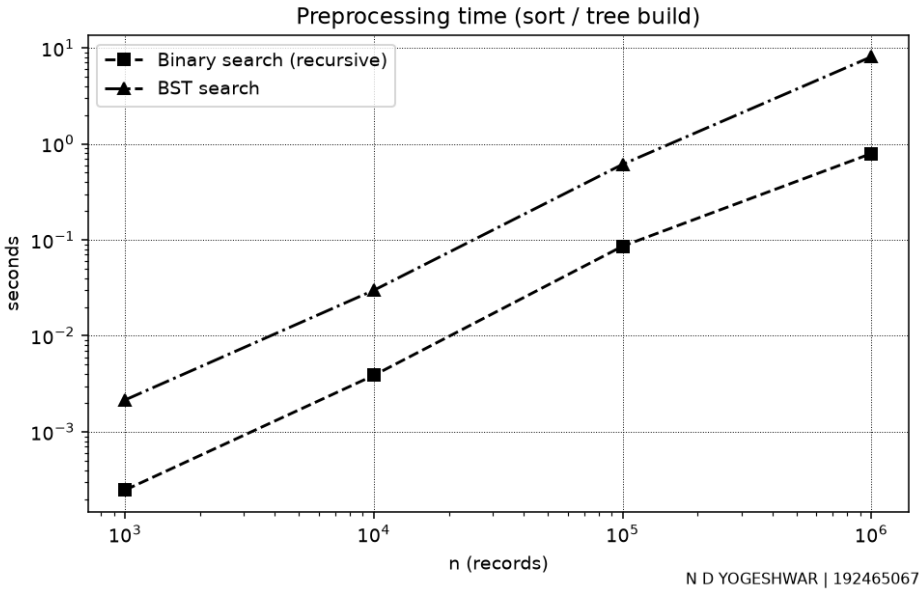


Figure 9.7: Preprocessing time (logarithmic axes)

Task 10. Theory Compared with Experiment

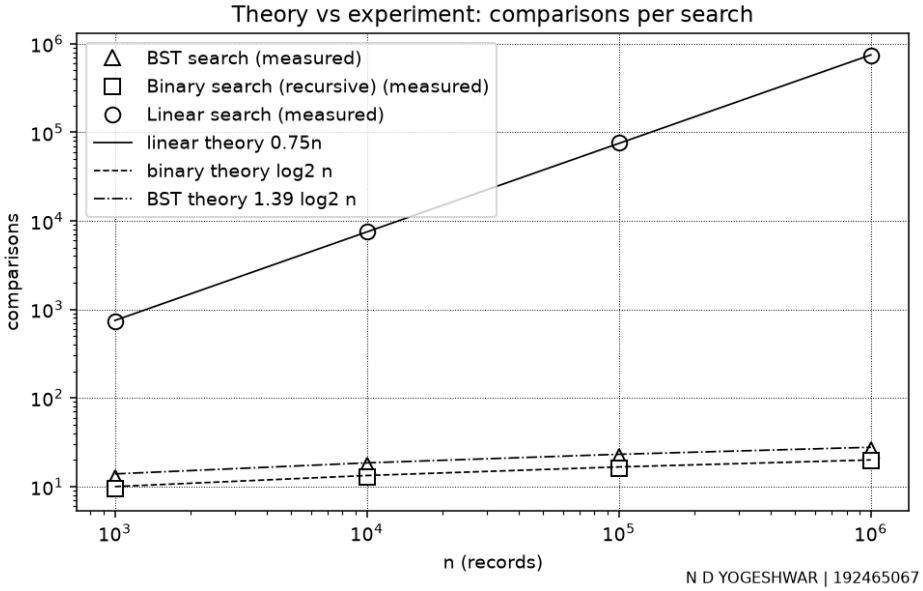


Figure 10.1: Measured comparisons (markers) and theoretical curves (lines)

The numbers of comparisons agree with the theory. Half of the queries are present keys and half are absent, so the average for linear search should be (n/2 + n)/2 = 0.75n, and the measured values are 739 for n = 1,000 and 737,490 for n = 1,000,000, a growth of 998 times for a database that is 1,000 times larger. Binary search went from 9.4 to 19.4 comparisons while $\log_2 n$ goes from 9.97 to 19.93, and the tree went from 12.5 to 25.6, always close to 1.3 times the binary search value. The measured heights of the tree were 21, 32, 44, 52 for the four sizes, which is more than $\log_2 n$, as is expected for a tree built from random input.

The running times do not follow the theory as exactly as the counts. The time of linear search grew 2993 times, not 1,000 times. One possible reason is that a list of a million integers no longer fits in the processor cache, so each step becomes slower, and another is background activity on the

machine. The tree was faster per search than the recursive binary search at one million records (7.7 µs against 14.7 µs) although it makes more comparisons (25.6 against 19.4). The probable cause is the cost of function calls in Python, since every step of the recursive binary search is a function call whereas a step in the tree search is only one iteration of a loop. The iterative binary search used in Task 12 removes a large part of this difference. Small irregularities in the table, such as the binary search time at 100,000 records, are caused by timing noise, because 202 searches take only a few milliseconds. The comparison counts are free of this noise, and therefore they were used to confirm the growth rates.

Binary search and the tree both scale much better than linear search because their work grows with log n. Between these two, binary search needs fewer comparisons and a much cheaper preprocessing (0.79 s against 8.06 s at one million records), whereas the tree gave slightly faster searches in Python. The preprocessing times agree with n log n: from 100,000 to 1,000,000 records the sorting time grew 9.2 times and the tree building time 13.2 times, while n log n predicts 12.0. The tree grows more slowly than predicted because its nodes are scattered in memory.

Task 11. Final Design Recommendation

Binary search on a sorted array is recommended for the student record system.

From the theory, the worst-case cost of a search is exactly floor($\log_2$ n) + 1 comparisons, so an unfavourable arrangement of the data cannot make a search slower, and the only extra cost is the O(n log n) sort. In the experiment at 1,000,000 records the sort took 0.79 s, after which a search took about 14.7 µs with at most 20 comparisons, whereas a linear search took about 302 ms and up to 1,000,000 comparisons. The sort costs as much as roughly 3 linear searches, so the sorted array is ahead from the fourth search onward, and a university database is certainly searched more often than that.

The tree was not chosen for the following reasons. Its searches were slightly faster in the Python tests, but building it took 8.1 s, which is about 10 times the sorting time. It needs a node object with two pointers for every record and therefore much more memory than a list of numbers. Its worst case is n if the records arrive in sorted order, which is quite possible when registration numbers are issued in sequence, so it offers no guarantee comparable to that of binary search. Linear search is acceptable for 1,000 records, where a search takes about 0.1 ms and no preprocessing is needed, but it needs about 22 ms at 100,000 records and about 302 ms at 1,000,000 records.

The main limitation of the sorted array is that it has to stay sorted. Inserting a record into the middle moves about half of the array, so many small insertions are expensive, and this is handled by batching the updates in Task 12. If records were added continuously in random order, a balanced tree such as an AVL tree or a B-tree would be better than the plain tree used here. The solution remains practical as the database grows, because a thousand times more data adds only about ten comparisons, and even ten million records would need about 23.

Task 12. Day-4 Design Challenge

12.A Impact analysis

The workload is now defined: 50,000 searches per hour, about 14 per second or 1.2 million per day, on 1,000,000 records, and new records are added only once a day. In the original problem the

number of searches was not known, so only the cost of one search was compared with the cost of preprocessing. Now the number of searches q is known and very large, and this changes the assumption on which preprocessing is judged. A one-time cost is shared by 1.2 million searches a day and becomes small, whereas a cost paid on every search, such as the 300 ms of linear search, is multiplied by 50,000. Because updates are rare, the weak point of the sorted array, which is slow insertion, hardly matters, since it has to be repaired only once a day and in a single batch.

12.B Evaluation of the original design

The original recommendation was binary search on a sorted array. It was measured with 50,000 searches (25,000 stored and 25,000 absent keys) on 1,000,000 records. The recursive version took 0.72 s and the iterative version 0.50 s. As the hour has 3,600 s, this is 0.014 percent of the available time, which is easily acceptable, and the sort of 1.04 s is needed only once a day. Linear search would need about 14,036 s for the same searches (estimated from a smaller number of measured searches), which is about 3.9 hours of work for every hour of load, so it can never keep up. The tree needed 0.46 s for the searches but 10.2 s to be built. A hash table needed only 0.037 s, about 13 times less than iterative binary search, with a build time of 0.27 s. Another approach can therefore be faster, but binary search is already far faster than the requirement.

12.C Adaptation: the design is modified

The sorted array with binary search is retained, with two changes. First, the daily update no longer sorts the whole array again. The new records of the day (1,000 were simulated) are sorted separately and merged with the existing sorted array in one pass, which costs $O(n + m)$ in place of $O(n \log n)$. The measured times were 1.064 s for a complete sort and 0.083 s for the merge. Second, the iterative form of the search is used, because it avoids a function call at each step (14.4 µs against 10.0 µs per search). The search remains $O(\log n)$, and the daily update changes from $O(n \log n)$ to $O(n + m \log m)$. If only exact lookups of registration numbers were ever needed, a hash index could be added later, with expected $O(1)$ lookup.

```
C:\DAA-A1\src>  python task12_challenge.py
```

```
======================================================================
Reg No: 192465067 | Name: N D YOGESHWAR
Date/Time: 05-10-2026 23:25:02
TASK 12 - CHALLENGE: 50,000 searches on 1,000,000 records
======================================================================
50,000 queries built (25,000 found, 25,000 not found)
load: 50,000 per hour = 13.9 searches/second

results agree (found counts): binary=25,000 iterative=25,000 bst=25,000 hash=25,000 expected=25,000

design                                    prep (s)  50k searches (s)  us/search
Linear search (estimated from 200 queries)      0.0000        14035.6566  280713.13
Binary search - original (recursive)            1.0424            0.7188      14.38
Binary search - iterative                       1.0424            0.4988       9.98
BST search                                     10.2174            0.4558       9.12
Hash table (set) - revised index                0.2707            0.0373       0.75

Daily update: 1000 new records arrive once per day
  sorted array: full re-sort            1.0639 s
  sorted array: merge sorted batch      0.0833 s
  BST: insert 1,000 records           0.0908 s
  hash table: insert 1,000 records    0.0004 s

Searches per day at this rate: 1,200,000
  Linear search (estimated from 200 queries)   total search time for a day = 336855.76 s
  Binary search - original (recursive)         total search time for a day = 17.25 s
  Binary search - iterative                    total search time for a day = 11.97 s
  BST search                                   total search time for a day = 10.94 s
  Hash table (set) - revised index             total search time for a day = 0.89 s

results saved to results/task12_results.csv
```

```
N D YOGESHWAR | Reg No 192465067
```

Figure 12.1: Benchmark of 50,000 searches on 1,000,000 records with the daily update costs

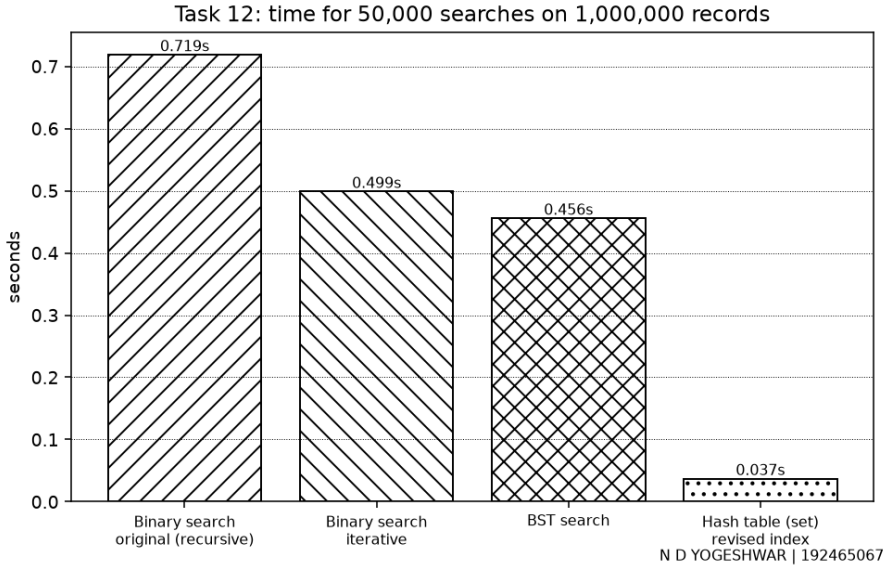


Figure 12.2: Time for 50,000 searches

12.D Defence of the decision

Design	Preprocessing (s)	50,000 searches (s)	Search time for one day (s)
Linear search (estimated from 200 queries)	0.000	14,035.657	336,855.8
Binary search - original (recursive)	1.042	0.719	17.3

Design	Preprocessing (s)	50,000 searches (s)	Search time for one day (s)
Binary search - iterative	1.042	0.499	12.0
BST search	10.217	0.456	10.9
Hash table (set) - revised index	0.271	0.037	0.9

Table 12.1: Results of the Task 12 benchmark. The linear search row is estimated from a smaller number of measured searches, since 50,000 linear searches would take several hours.

The decision follows from four quantities. As regards the frequency of searches, 14 searches per second can be served by any design that answers in microseconds, and only linear search fails, being about 28,138 times slower than iterative binary search. As regards preprocessing, the sort needs about 1.0 s a day against about 12 s of searching, and with merging only 0.08 s. As regards the size of the data, binary search needs at most 20 comparisons for 1,000,000 records, while the tree needs 10 s to be built, which makes it the worst choice when the structure has to be rebuilt. As regards trade-offs, the hash table is fastest for single lookups but cannot answer ordered or range questions, for example all the students of one batch, and it uses more memory, whereas the sorted array supports these questions and is the smaller structure. Since the system needs far less speed than binary search provides, the simplest design with a guaranteed worst case is kept. The original solution is therefore modified and not replaced.

Conclusion

Three search strategies were designed, analysed mathematically and tested on datasets from 1,000 to 1,000,000 records. The derived costs, n/2 for linear search, $\log_2 n$ for binary search and about $1.39 \log_2 n$ for a tree built from random input, were confirmed by the measured comparison counts. Binary search on a sorted array gave the best balance of preprocessing cost, guaranteed worst case and memory, and it was kept for the changed workload of Task 12 with a daily merge of new records.

References

T. H. Cormen, C. E. Leiserson, R. L. Rivest and C. Stein, Introduction to Algorithms, 3rd edition, MIT Press, 2009.

A. Levitin, Introduction to the Design and Analysis of Algorithms, 3rd edition, Pearson, 2012.

D. E. Knuth, The Art of Computer Programming, Volume 3: Sorting and Searching, 2nd edition, Addison-Wesley, 1998.